

Optimale Programmausführung mit lokaler Information

LIS[∞]: Local Information Scheduling mit globaler Perspektive

Erweiterte Fassung mit Model Checking und Temporaler Logik

Stephan Epp

17. September 2026

Inhaltsverzeichnis

1	Einführung	3
1.1	Motivation und Problemstellung	3
1.2	Struktur und Beiträge	3
2	Grundlagen und Definitionen	4
2.1	Lokale Information und Pulsleitung-Architektur	4
3	Lokale Konsistenz und Optimalität	5
3.1	Definition lokaler Konsistenz	5
3.2	Kernlemma: Lokale Konsistenz	5
3.3	Globaloptimalität aus Lokalität	5
4	Hierarchische Register-Lokalisierung	6
4.1	Multi-Layer Cache-Modell	6
4.2	Optimalität der Hierarchie	6
5	Model Checking und Temporale Logik für Vorhersage in LIS[∞]	6
5.1	Einleitung: Warum Model Checking für LIS [∞] ?	6
5.2	Temporal Logic: Grundlagen	7
5.2.1	Linear Temporal Logic (LTL)	7
5.2.2	Computation Tree Logic (CTL)	8
5.3	Kripke-Strukturen und Transitionssysteme für LIS [∞]	8
5.4	Model Checking Algorithmen für LIS [∞]	8
5.4.1	LTL Model Checking	8
5.4.2	CTL Model Checking	9
5.4.3	Komplexitätsanalyse	10
5.5	Vorhersage durch Model Checking	10
5.5.1	Vorhersage-Problem	10
5.5.2	Vorhersage und Lokalität	11

5.6	Spezifikation von Sicherheits- und Lebendigkeit-Eigenschaften	11
5.6.1	Safety-Eigenschaften	11
5.6.2	Liveness-Eigenschaften	12
5.7	Invarianten für LIS [∞] -Systeme	12
5.8	Verifizierung praktischer Programme	12
6	Formale Verifikation (Aktualisiert)	13
6.1	Temporale Logik und Invarianten (erweitert)	13
7	Praktische Anwendung und Effizienzanalyse	13
7.1	Optimierungsbeispiel: LED-Ansteuerung	13
7.2	Komplexitätsanalyse	14
8	Vergleich mit klassischen Ansätzen	14
8.1	LIS [∞] vs. globale Programmplanung	14
9	Schlussfolgerungen und Ausblick	14
9.1	Schlussfolgerungen	14
9.2	Ausblick	15
9.2.1	Theoretische Erweiterungen	15
9.2.2	Praktische Entwicklungen	15
9.2.3	Neuartige Anwendungen	16
9.2.4	Philosophische Perspektiven	16
9.3	Abschließende Bemerkung	16

1 Einführung

Diese Arbeit präsentiert LIS^∞ (Local Information Scheduling mit globaler Perspektive), einen neuartigen Ausführungsrahmen für Programmabarbeitung, der die fundamentale Dichotomie zwischen lokaler Verarbeitung auf der Ebene der Register und globaler Systemoptimierung durch die bewusste Nutzung von Pulsleitung-Architektur auflöst.

Die zentrale These dieser Arbeit lautet: **Optimale Programmausführung erfordert nicht die globale Voraussicht aller Konsequenzen, sondern die lokale Konsistenz der Informationsverarbeitung in jeder Abarbeitungsphase, kombiniert mit einem deterministischen, globalen Perspektiv-Regelsystem.**

1.1 Motivation und Problemstellung

Der Mensch steht sich oft selbst im Wege, weil er sich der Konsequenzen seines Handelns nicht bewusst ist oder diese nicht akzeptiert. Die Natur hingegen demonstriert kontinuierlich das Prinzip der *unbedingten Konsequenz*: Die Sonne scheint immer, die Erdanziehungskraft wirkt beständig, und auf dem Boden der Erde ermöglicht die geordnete Begegnung von Objekten und Menschen eine stabile, nutzbare Welt.

Analoges gilt für Computer: Sie arbeiten ausschließlich mit **lokaler Information**. Ein Register speichert lokal einen Wert, eine Pulsleitung übermittelt lokal ein Bit (0V oder 3,3V), ein Adressbus leitet lokal eine Adresse durch. Kein Teil der CPU verfügt über globales Wissen. Dennoch führt die konsistente Verkettung dieser lokalen Operationen zu global korrekter, optimaler Ausführung.

Die Frage lautet daher nicht: *“Wie kann der Kontroller oder die CPU global sehen und entscheiden?”* sondern vielmehr: *“Wie gestalten wir die lokale Informationsverarbeitung und die Pulsleitung-Architektur so, dass globale Optimalität als Konsequenz lokaler Konsistenz entsteht?”*

1.2 Struktur und Beiträge

Diese Arbeit leistet folgende Beiträge:

1. **Formalisierung lokaler Information:** Wir definieren ein mathematisches Modell, das Register, Pulsleitungen und den Kontroller als Einheit lokaler Informationsprozessoren beschreibt.
2. **Determinismus durch Konsistenz:** Wir beweisen, dass lokale Konsistenz (Lemma 1) notwendig und hinreichend für globale Optimalität ist.
3. **Architektur-Optimierungstheorem:** Wir leiten ein Fundamentalsatz (Satz ??) her, der zeigt, dass jede lokale Pulsoperation in einem Register-basierten System unter dem Prinzip der Konsequenz-Determinismus optimal ist.
4. **Programmabarbeitungs-Algorithmus:** Wir definieren LIS^∞ , einen Algorithmus, der die lokalen Informationen durch Register-Hierarchien und Pulsleitung-Sequenzen optimal orchestriert.
5. **Model Checking und Temporale Logik: Neu:** Wir integrieren formale Verifikationsmethoden durch Temporale Logik (LTL und CTL) und Model Checking, um Vorhersagen über Programmverhalten zu machen (Kapitel 5).

6. **Praktische Effizienz:** Wir demonstrieren die Anwendbarkeit auf moderne Mikrocontroller und embedded systems.

2 Grundlagen und Definitionen

2.1 Lokale Information und Pulsleitung-Architektur

Definition 1 (Pulsleitung). Eine **Pulsleitung** ℓ ist eine physikalische oder abstrakte Verbindung, über die eine Information in Form eines Pulses $p \in \{0, 1\}$ übertragen wird:

$$p(\ell, t) = \begin{cases} 0 & \text{falls Spannungspegel} = 0V \text{ (Low)} \\ 1 & \text{falls Spannungspegel} = 3.3V \text{ (High)} \end{cases}$$

wobei t die Zeitachse bezeichnet. Eine Sequenz von Pulsen auf ℓ konstituiert eine **Nachricht**.

Definition 2 (Lokale Information). Eine **lokale Information** I_{lok} ist ein Datum, das in genau einem Register, genau einer Pulsleitung oder genau einem Controller-Zustand zu einer gegebenen Zeit t existiert:

$$I_{\text{lok}}(t) = I_{\text{reg}} \cup I_{\text{puls}} \cup I_{\text{ctrl}},$$

wobei: $- I_{\text{reg}}$: Wert in einem Register (z.B. Akkumulator, Pointer, Zustandsvariable), $- I_{\text{puls}}$: Bit auf einer Pulsleitung, $- I_{\text{ctrl}}$: Zustand des Controller-Automaten.

Zentrales Prinzip: Kein Prozess in der CPU hat gleichzeitiges Zugriff auf alle I_{lok} des Systems. Nur der nächste Prozess in der Ausführungskette erhält Zugriff.

Definition 3 (Bus-System). Ein **Bus-System** $\mathcal{B}$ ist ein Tripel

$$\mathcal{B} = (\mathcal{A}, \mathcal{D}, \mathcal{C}),$$

bestehend aus: $- \text{Adressbus } \mathcal{A} = (\ell_{a,1}, \dots, \ell_{a,m})$: m Pulsleitungen für Adressen, $- \text{Datenbus } \mathcal{D} = (\ell_{d,1}, \dots, \ell_{d,n})$: n Pulsleitungen für Daten, $- \text{Kontrollbus } \mathcal{C} = (\ell_{c,1}, \dots, \ell_{c,k})$: k Pulsleitungen für Steuersignale (Read, Write, Enable).

Eine Nachricht auf $\mathcal{A}$ zur Zeit t ist ein m -Bit-Vektor $\mathbf{a}(t) = (p(\ell_{a,1}, t), \dots, p(\ell_{a,m}, t)) \in \{0, 1\}^m$.

Definition 4 (Register-Zustand). Ein **Register-Zustand** eines Systems ist das Tupel

$$\mathcal{R}(t) = (r_1(t), r_2(t), \dots, r_s(t)) \in \{0, 1\}^w \times \dots \times \{0, 1\}^w,$$

wobei $r_i(t) \in \{0, 1\}^w$ der Wert des i -ten Registers (Wortbreite w) zur Zeit t ist. Die Menge aller möglichen Register-Zustände sei $\mathbb{R} = \{0, 1\}^{s \cdot w}$.

Definition 5 (Controller-Automat). Der **Controller-Automat** $\mathcal{K}$ ist ein determinierter Transitionssystem (DEA):

$$\mathcal{K} = (\mathbb{Q}, \Sigma_{\text{puls}}, \delta_{\mathcal{K}}, q_0),$$

wobei: $- \mathbb{Q}$: endliche Menge von Zuständen, $- \Sigma_{\text{puls}} = \{0, 1\}^{|\mathcal{A}|+|\mathcal{D}|+|\mathcal{C}|}$: Eingabealphabet (alle Pulsleitung-Kombinationen), $- \delta_{\mathcal{K}} : \mathbb{Q} \times \Sigma_{\text{puls}} \rightarrow \mathbb{Q}$: Übergangsfunktion, $- q_0 \in \mathbb{Q}$: Initialzustand.

Der Controller arbeitet in **Zyklen**: In jedem Takt t liest der Controller alle Pulsleitungen (lokale Information aus Umgebung), wechselt seinen Zustand nach $\delta_{\mathcal{K}}$, und emittiert neue Steuersignale auf $\mathcal{C}$.

Definition 6 (Programmabarbeitung). Eine **Programmabarbeitung** ist eine endliche oder unendliche Sequenz von Transitionen:

$$(q_0, \mathcal{R}(0)) \xrightarrow{\text{Puls}_1} (q_1, \mathcal{R}(1)) \xrightarrow{\text{Puls}_2} (q_2, \mathcal{R}(2)) \cdots,$$

wobei in jedem Schritt ein Puls auf eine oder mehrere Pulsleitungen erfolgt, der Register-Zustand und Kontroller-Zustand konsistent aktualisiert werden.

3 Lokale Konsistenz und Optimalität

3.1 Definition lokaler Konsistenz

Definition 7 (Lokale Konsistenz). Ein Register-Zustand $\mathcal{R}(t)$ und Kontroller-Zustand $q(t)$ sind **lokal konsistent**, wenn folgende Bedingungen gelten:

1. **Eindeutigkeit:** Jedes Register hat zu t genau einen wohldefinierten Wert.
2. **Kausalität:** Wenn Register $r_i(t)$ vom Register $r_j(t')$ abhängt, dann $t' < t$.
3. **Determinismus:** Der Zustand $(q(t), \mathcal{R}(t))$ führt über die Übergangsfunktion zu genau einem Nachfolgezustand $(q(t+1), \mathcal{R}(t+1))$.

3.2 Kernlemma: Lokale Konsistenz

Lemma 1 (Lokale Konsistenz impliziert Determinismus). Wenn ein System in allen Zeitpunkten t lokal konsistent ist, dann ist die Programmabarbeitung vollständig determiniert.

Beweis. Gegeben: System ist in allen Zeitpunkten lokal konsistent.

Schritt 1 (Eindeutigkeit): Nach Definition sind alle Register eindeutig definiert.

Schritt 2 (Determinismus der Übergangsfunktion): Aus der Eindeutigkeit und Definition folgt, dass die Übergangsfunktion δ_K mit eindeutigem Input einen eindeutigen Output produziert.

Schritt 3 (Induktion): Angenommen, Zustand $(q(t), \mathcal{R}(t))$ ist eindeutig. Nach Definition 3 (Determinismus) folgt $(q(t+1), \mathcal{R}(t+1))$ ist eindeutig. Per Induktion ist die gesamte Ausführungssequenz determiniert. $\square$

3.3 Globaloptimalität aus Lokalität

Satz 1 (Lokalität impliziert Optimalität). Wenn eine Programmabarbeitung lokal konsistent ist, dann existiert keine alternative Abarbeitungssequenz, die weniger Pulse benötigt und zum gleichen Endzustand führt.

Beweis. **Beweis durch Widerspruch:**

Angenommen, es gäbe eine alternative Sequenz τ' mit weniger Pulsen als die konsistente Sequenz τ .

Sei π_1 der erste Schritt in τ' , der sich von τ unterscheidet. In τ wird ein Schritt s_i ausgeführt, in τ' wird s_j ausgeführt ($i \neq j$).

Fall 1: s_j ist unabhängig von s_i . Dann können beide in τ parallel ausgeführt werden (aber wir haben Pulsleitung-Serialität, also nur sequenziell). Verzicht auf s_i verfälscht den Endzustand.

Fall 2: s_j hängt von s_i ab. Dann kann s_j nicht vorher ausgeführt werden (Kausalitätsverletzung).

Fall 3: s_i hängt von s_j ab. Dann ist τ' möglich, aber s_i muss später wiederholt werden, um den Endzustand zu erreichen. Das kostet zusätzliche Pulse.

Alle Fälle führen zu Widerspruch mit der Annahme, dass τ' weniger Pulse benötigt. Daher ist τ optimal. $\square$

4 Hierarchische Register-Lokalisierung

4.1 Multi-Layer Cache-Modell

Definition 8 (Register-Hierarchie). Eine **Register-Hierarchie** ist ein Tupel von Speicher-Layern

$$(L_1, L_2, \dots, L_k),$$

wobei: – L_i schneller, aber kleiner ist als L_{i+1} , – Jeder Layer ist eine Menge von Registern, – Ein Wert kann von L_i zu L_{i+1} kopiert werden.

4.2 Optimalität der Hierarchie

Satz 2 (Hierarchische Optimalität). Für eine Register-Hierarchie $(L_1, L_2, \dots, L_k)$ mit Latenzzeiten $\text{Lat}(L_i) = (i - 1) \cdot c$ (linear) gelten:

1. **Exklusivität:** Ein Wert existiert zu Zeit t in höchstens einem Layer,
2. **Konsistenz:** Wenn L_i von L_{i+1} liest, bleibt der Wert korrekt verfügbar,
3. **Optimalität:** Die Gesamtanzahl von Pulsen ist minimal, wenn häufige Zugriffe in schnelleren Layern stattfinden.

Beweis. (1) **Exklusivität:** Nach Definition lokaler Information existiert jedes Datum an genau einem Ort zu genau einer Zeit. Ein Wert kann nicht gleichzeitig in L_1 und L_2 sein. (Dies ist ein Konsistenzmodell-Postulat, das in realen Systemen durch Cache-Kohärenz-Protokolle implementiert wird.)

(2) **Konsistenz:** Wenn L_i vom Layer L_{i+1} liest, entsteht eine Abhängigkeit. Solange diese Abhängigkeit aktiv ist, muss der Wert korrekt verfügbar bleiben. Dies ist ein Invarianten-Erhaltungs-Argument.

(3) **Optimalität:** Folgt direkt aus Satz 1. $\square$

5 Model Checking und Temporale Logik für Vorhersage in LIS^∞

5.1 Einleitung: Warum Model Checking für LIS^∞ ?

Während die bisherigen Kapitel die theoretischen Grundlagen von LIS^∞ gelegt haben, stellt sich eine praktische Frage: **Wie können wir garantieren, dass ein konkretes Programm in LIS^∞ die erwünschten Eigenschaften erfüllt?**

Die Antwort liegt in der **formalen Verifikation durch Model Checking und Temporale Logik**. Diese Kapitel integriert:

1. Fundamentale temporale Logiken (LTL und CTL) für die Spezifikation von Programmeigenschaften,
2. Formale Model Checking Algorithmen zur Verifikation dieser Eigenschaften,
3. Vorhersage-Mechanismen, die aus der Temporalen Logik abgeleitet sind,
4. Praktische Anwendungen auf LIS[∞]-Programme.

5.2 Temporal Logic: Grundlagen

5.2.1 Linear Temporal Logic (LTL)

Definition 9 (LTL Syntax). Die Syntax von LTL ist induktiv definiert:

$$\varphi ::= \text{true} \mid p \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \mathbf{X}\varphi \mid \mathbf{F}\varphi \mid \mathbf{G}\varphi \mid \varphi_1 \mathbf{U}\varphi_2,$$

wobei:

- p eine atomare Proposition ist,
- $\mathbf{X}\varphi$ (neXt): φ gilt im nächsten Zustand,
- $\mathbf{F}\varphi$ (Finally): φ gilt irgendwann in der Zukunft,
- $\mathbf{G}\varphi$ (Globally): φ gilt in jedem Zustand,
- $\varphi_1 \mathbf{U}\varphi_2$ (Until): φ_1 gilt, bis φ_2 gilt.

Definition 10 (LTL Semantik). Gegeben ist ein Pfad (Ausführungstrace) $\pi = s_0, s_1, s_2, \dots$ Zustände. Sei $\pi^i = s_i, s_{i+1}, \dots$ der Suffix-Pfad ab Position i .

Die Erfüllungsrelation $\pi \models \varphi$ ist definiert als:

$$\pi \models p \iff p \in L(s_0), \tag{1}$$

$$\pi \models \neg\varphi \iff \pi \not\models \varphi, \tag{2}$$

$$\pi \models \varphi_1 \wedge \varphi_2 \iff \pi \models \varphi_1 \text{ und } \pi \models \varphi_2, \tag{3}$$

$$\pi \models \mathbf{X}\varphi \iff \pi^1 \models \varphi, \tag{4}$$

$$\pi \models \mathbf{F}\varphi \iff \exists i \geq 0 : \pi^i \models \varphi, \tag{5}$$

$$\pi \models \mathbf{G}\varphi \iff \forall i \geq 0 : \pi^i \models \varphi, \tag{6}$$

$$\pi \models \varphi_1 \mathbf{U}\varphi_2 \iff \exists j \geq 0 : \pi^j \models \varphi_2 \text{ und } \forall 0 \leq i < j : \pi^i \models \varphi_1. \tag{7}$$

Beispiel 1 (LTL Formeln für LIS[∞]-Programme). 1. $\Box(\text{Register konsistent})$: Das Register ist in jedem Zustand konsistent,

2. $\Diamond(\text{HALT erreicht})$: Das Programm erreicht irgendwann den HALT-Befehl,
3. $\Box(\text{Instruktion } i \rightarrow \Diamond(\text{Instruktion } j))$: Nach jeder Ausführung von Instruktion i folgt irgendwann Instruktion j ,
4. $\Box\neg(\text{Datenverlust})$: Es gibt niemals Datenverlust.

5.2.2 Computation Tree Logic (CTL)

Definition 11 (CTL Syntax). CTL erweitert LTL um Pfad-Quantoren. Die Syntax ist:

$$\varphi ::= \text{true} \mid p \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \mathbf{AX}\varphi \mid \mathbf{EX}\varphi \mid \mathbf{AF}\varphi \mid \mathbf{EF}\varphi \mid \mathbf{AG}\varphi \mid \mathbf{EG}\varphi,$$

wobei:

- $\mathbf{A}\psi$: auf allen Pfaden ($\forall$),
- $\mathbf{E}\psi$: auf existierenden Pfaden ($\exists$).

5.3 Kripke-Strukturen und Transitionssysteme für LIS^∞

Definition 12 (Kripke-Struktur für LIS^∞). Eine **Kripke-Struktur** M_{LIS} für ein LIS^∞ -System ist ein Tupel

$$M_{\text{LIS}} = (S, S_0, \rightarrow, L),$$

wobei:

- $S = \mathbb{Q} \times \mathbb{R}$ die Menge aller Zustände ist (Kontroller-Zustand $\times$ Register-Zustand),
- $S_0 \subseteq S$ die Menge der Initialzustände ist,
- $\rightarrow \subseteq S \times S$ die Transitionsrelation ist (definiert durch δ_K und Register-Updates),
- $L : S \rightarrow \mathcal{P}(AP)$ die Labeling-Funktion ist, die jeden Zustand mit erfüllten atomaren Propositionen (AP) kennzeichnet.

Definition 13 (Atomare Propositionen für LIS^∞). Typische atomare Propositionen für LIS^∞ -Programme sind:

1. Register-Bedingungen: $\text{reg_0} = k, \text{reg_1} > 100$,
2. Kontroller-Zustände: $q = q_{\text{init}}, q = q_{\text{halt}}$,
3. Pulsleitung-Zustände: $\text{addr_bus} = 0x4001, \text{ctrl_bus} = \text{write}$,
4. Programm-Zustände: $\text{instr_at}(i)$ (Instruktion i wird gerade ausgeführt).

5.4 Model Checking Algorithmen für LIS^∞

5.4.1 LTL Model Checking

Satz 3 (LTL Model Checking Problem). Gegeben eine Kripke-Struktur M_{LIS} und eine LTL-Formel φ . Das **LTL Model Checking Problem** ist:

$$M_{\text{LIS}} \models \varphi \iff \forall s_0 \in S_0 : \forall \pi \text{ Pfade ab } s_0 : \pi \models \varphi.$$

Das Problem ist NP-vollständig für allgemeine LTL-Formeln, aber polynomial für Formeln in speziellen Subklassen.

Algorithm 1 LTL Model Checking (Automatentheorie-Ansatz)

Require: Kripke-Struktur M_{LIS} , LTL-Formel φ

Ensure: Ergebnis: TRUE wenn $M_{\text{LIS}} \models \varphi$, sonst FALSE (mit Gegenbeispiel)

- 1: Konstruiere Büchi-Automat $\mathcal{B}_{\neg\varphi}$ für $\neg\varphi$
 - 2: Konstruiere Produktautomat $M_{\text{LIS}} \times \mathcal{B}_{\neg\varphi}$
 - 3: Prüfe auf erreichbare akzeptierende Zyklen mittels Tiefensuche
 - 4: **if** akzeptierender Zyklus gefunden **then**
 - 5: **return** FALSE (mit Zyklus als Gegenbeispiel)
 - 6: **else**
 - 7: **return** TRUE (Formel erfüllt)
 - 8: **end if**
-

Lemma 2 (Korrektheit LTL Model Checking). Algorithmus 1 ist korrekt.

Beweis. **Korrektheit:**

Ein erreichbarer akzeptierender Zyklus im Produktautomaten entspricht einem Pfad π ab s_0 mit $\pi \models \neg\varphi$, also $\pi \not\models \varphi$. Dies widerlegt $M_{\text{LIS}} \models \varphi$.

Vollständigkeit:

Wenn kein akzeptierender Zyklus existiert, dann kann kein Pfad $\neg\varphi$ erfüllen. Also erfüllen alle Pfade φ . Daher $M_{\text{LIS}} \models \varphi$. $\square$

5.4.2 CTL Model Checking

Satz 4 (CTL Model Checking Problem). Das CTL Model Checking Problem ist entscheidbar in polynomialer Zeit $O(|M| \cdot |\varphi|)$.

Algorithm 2 Kripke Model Checking für CTL

Require: Kripke-Struktur M_{LIS} , CTL-Formel φ

Ensure: Menge $\text{Sat}(\varphi) \subseteq S$ aller Zustände, die φ erfüllen

```
1: if  $\varphi$  ist Proposition  $p$  then
2:   return  $\{s \in S : p \in L(s)\}$ 
3: else if  $\varphi = \neg\psi$  then
4:   return  $S \setminus \text{ModelCheck}(M, \psi)$ 
5: else if  $\varphi = \psi_1 \wedge \psi_2$  then
6:   return  $\text{ModelCheck}(M, \psi_1) \cap \text{ModelCheck}(M, \psi_2)$ 
7: else if  $\varphi = \mathbf{AX}\psi$  then
8:   return  $\{s : \forall s' \in \text{succ}(s) : s' \in \text{ModelCheck}(M, \psi)\}$ 
9: else if  $\varphi = \mathbf{EX}\psi$  then
10:  return  $\{s : \exists s' \in \text{succ}(s) : s' \in \text{ModelCheck}(M, \psi)\}$ 
11: else if  $\varphi = \mathbf{AF}\psi$  then
12:   $Z_0 \leftarrow \text{ModelCheck}(M, \psi)$ 
13:   $Z_{i+1} \leftarrow Z_i \cup \{s : \forall s' \in \text{succ}(s) : s' \in Z_i\}$ 
14:  repeat until Fixpunkt erreicht
15:  return  $Z_{\text{fix}}$ 
16: else if  $\varphi = \mathbf{EG}\psi$  then
17:   $Z_0 \leftarrow \text{ModelCheck}(M, \psi)$ 
18:   $Z_{i+1} \leftarrow Z_i \cap \{s : \exists s' \in \text{succ}(s) : s' \in Z_i\}$ 
19:  repeat until Fixpunkt erreicht
20:  return  $Z_{\text{fix}}$ 
21: end if
```

Lemma 3 (Korrektheit CTL Model Checking). Algorithmus ?? berechnet korrekt die Menge aller Zustände, die φ erfüllen.

Beweis. Die Korrektheit folgt aus der semantischen Definition von CTL und den Fixpunkt-Eigenschaften der temporalen Operatoren. Die Fixpunkt-Iterationen konvergieren, weil S endlich ist. $\square$

5.4.3 Komplexitätsanalyse

Theorem 1 (Komplexität von Model Checking). 1. **LTL Model Checking:** PSPACE-vollständig im allgemeinen Fall. Für Formeln ohne verschachtelte Negationen: polynomial.

2. **CTL Model Checking:** P-komplett. Lineare Komplexität in der Struktur und exponentiell in der Formel (aber Formeln sind typisch kurz).

3. **Für LIS $^\infty$:** Mit $|S| = 2^{s \cdot w + |\mathbb{Q}|}$ Zuständen:

- CTL: $O(2^{s \cdot w} \cdot |\mathbb{Q}| \cdot |\varphi|)$ Zeit,
- LTL: mit Büchi-Automat zusätzlich exponentiell in $|\varphi|$.

5.5 Vorhersage durch Model Checking

5.5.1 Vorhersage-Problem

Definition 14 (Vorhersage in LIS $^\infty$). Gegeben:

- Ein LIS^∞ -Programm P ,
- Aktueller Zustand $s_{\text{curr}} = (q_{\text{curr}}, \mathcal{R}_{\text{curr}})$,
- Eine Zieleigenschaft φ (in LTL oder CTL).

Vorhersage-Frage: Wird die Eigenschaft φ aus dem aktuellen Zustand s_{curr} erfüllt?

Vorhersage-Antwort: Berechne die Teilmenge $T_\varphi \subseteq S$ aller Zustände, aus denen φ erfüllt wird. Prüfe dann $s_{\text{curr}} \in T_\varphi$.

Algorithm 3 Vorhersage-Algorithmus für LIS^∞

Require: LIS^∞ -System mit Kripke-Struktur M_{LIS} , aktueller Zustand s_{curr} , Formel φ

Ensure: Vorhersage: erfüllt oder verletzt

```

1:  $T_\varphi \leftarrow \text{ModelCheck}(M_{\text{LIS}}, \varphi)$  // Berechne alle Zustände, die  $\varphi$  erfüllen
2: if  $s_{\text{curr}} \in T_\varphi$  then
3:   return erfüllt
4: else
5:   Berechne Pfad von  $s_{\text{curr}}$  zum nächsten Zustand in  $T_\varphi$  (falls vorhanden)
6:   if solcher Pfad existiert then
7:     return verfolgbar in  $k$  Schritten
8:   else
9:     return nicht erreichbar
10:  end if
11: end if

```

5.5.2 Vorhersage und Lokalität

Satz 5 (Vorhersage respektiert lokale Konsistenz). Wenn aus dem aktuellen Zustand s_{curr} lokal konsistente Transitionen folgen, dann stimmt die Vorhersage mit der tatsächlichen Ausführung überein.

Beweis. Die Vorhersage wird durch die Kripke-Struktur M_{LIS} berechnet, die alle möglichen lokalen Transitionen enthält. Wenn s_{curr} zu einer Eigenschaft φ führen kann, wird dies von Model Checking erkannt. Deterministische Transitionen führen zu eindeutigen Vorhersagen. $\square$

5.6 Spezifikation von Sicherheits- und Lebendigkeit-Eigenschaften

5.6.1 Safety-Eigenschaften

Definition 15 (Safety). Eine Eigenschaft φ ist eine **Safety-Eigenschaft**, wenn:

$\neg\varphi$ kann nach endlich vielen Schritten erkannt werden.

In LTL: Safety-Eigenschaften sind von der Form $\mathbf{G}\psi$ (Globally ψ).

Beispiel 2 (Safety in LIS^∞). 1. $\mathbf{G}\neg(\text{Register_Konflikt})$: Niemals zwei Schreibvorgänge auf dasselbe Register,

2. $\mathbf{G}(\text{Speicher_korrekt})$: Speicher ist immer konsistent,

3. $\mathbf{G}\neg(\text{Div_by_Zero})$: Keine Division durch Null.

5.6.2 Liveness-Eigenschaften

Definition 16 (Liveness). Eine Eigenschaft φ ist eine **Liveness-Eigenschaft**, wenn jeder endliche Pfad eine Erweiterung auf einen unendlichen Pfad hat, der φ erfüllt.

In LTL: Liveness-Eigenschaften sind von der Form $\mathbf{F}\psi$ (Finally ψ).

Beispiel 3 (Liveness in LIS^∞). 1. $\mathbf{F}(\text{HALT})$: Das Programm terminiert irgendwann,
2. $\mathbf{G}(\text{Request} \rightarrow \mathbf{F}\text{Grant})$: Jede Anfrage wird irgendwann beantwortet,
3. $\mathbf{F}(\text{Speicher_vollst\"andig_geschrieben})$: Alle Daten werden irgendwann gespeichert.

5.7 Invarianten für LIS^∞ -Systeme

Definition 17 (LIS^∞ -Invarianten). Ein LIS^∞ -System erfüllt folgende Korrektheitsinvarianten:

1. $\Box(\text{Keine zwei Schreibvorg\"ange auf dasselbe Register})$ (Safety),
2. $\Box(\text{Jede Instruktion wird beendet})$ (Liveness),
3. $\Box(\text{Register-Zustand konsistent}) \Rightarrow \Box(\text{n\"achster Zustand existiert})$ (Determinismus),
4. $\Diamond(\text{HALT-Instruktion erreicht})$ oder $\Box\neg\text{HALT}$ (Terminierung oder Schleife).

Lemma 4 (Invarianten-Einhaltung). Jedes in LIS^∞ ausgeführte Programm mit lokal konsistenten Zustandsübergängen erfüllt alle vier Invarianten.

Beweis. (1) **Keine Konflikte:** Nach der Algorithmus-Struktur (While-Schleife sequenziell). Jedes Register wird in einer atomaren Operation aktualisiert.

(2) **Liveness:** Jede Instruktion führt zur Inkrementierung des Program Counters. Bei endlichem Programm terminiert die While-Schleife. Bei Schleifen wird der Liveness-Zustand periodisch erreicht.

(3) **Determinismus:** Folgt aus Lemma 1: Lokale Konsistenz impliziert Determinismus.

(4) **Terminierung:** Prüfung durch Kontrollflussgraph-Analyse. Wenn das Programm HALT erreichen kann, endet es. Ansonsten Schleife. $\square$

5.8 Verifizierung praktischer Programme

Beispiel 4 (Verifikation: LED-Ansteuerung mit temporaler Logik). Gegeben: Programm zur GPIO-LED-Ansteuerung (wie in Beispiel 5).

Zu verifizierende Eigenschaften:

1. $\mathbf{G}\neg(\text{GPIO_datenverlust})$: Keine Daten gehen verloren,
2. $\mathbf{F}(\text{LED_leuchtet})$: Die LED leuchtet irgendwann auf,
3. $\mathbf{X}(\text{LED_leuchtet})$ falls write_erfolgreich : Nach erfolgreichem Schreiben geht die LED an.

Verifikationsprozess:

- Modelliere das GPIO-System als Kripke-Struktur,
- Kodiere die Eigenschaften als LTL-Formeln,
- Wende LTL Model Checking an.

Ergebnis: Wenn Model Checking erfolgreich ist, sind die Eigenschaften bewiesen.

6 Formale Verifikation (Aktualisiert)

6.1 Temporale Logik und Invarianten (erweitert)

Um die Korrektheit von LIS^∞ umfassend zu verifizieren, nutzen wir die in Abschnitt 5 eingeführte Temporale Logik. Die Invarianten sind:

Definition 18 (Erweiterte Korrektheitsinvarianten). Ein LIS^∞ -System erfüllt Korrektheit, wenn folgende LTL-Formeln für alle Abarbeitungspfade gelten:

1. $\Box(\text{Keine zwei Schreibvorgänge auf dasselbe Register})$ (Safety),
2. $\Box(\text{Jede Instruktion wird beendet})$ (Liveness),
3. $\Box(\text{Register-Zustand konsistent}) \Rightarrow \Box(\text{nächster Zustand existiert})$ (Determinismus),
4. $\Diamond(\text{HALT-Instruktion erreicht})$ oder $\Box\neg\text{HALT}$ (Terminierung oder Schleife),
5. $\Box(\text{Keine Pulse kollidieren})$ (Pulsleitung-Integrität).

Lemma 5 (Invarianten-Einhaltung (erweitert)). LIS^∞ hält alle fünf Invarianten ein.

Beweis. Folgt aus den Beweisen in Lemma 4 mit der zusätzlichen Eigenschaft der Pulsleitung-Architektur: Da jede Pulsleitung durch den Controller kontrolliert wird und der Controller deterministisch arbeitet, können sich Pulse nicht kollidieren (ein Puls pro Zeittakt per Leitung). $\square$

7 Praktische Anwendung und Effizienzanalyse

7.1 Optimierungsbeispiel: LED-Ansteuerung

Wir demonstrieren LIS^∞ auf einem praktischen Beispiel aus der `cpu.md`.

Beispiel 5 (LED mit GPIO ansteuern). **Ziel:** Setze GPIO-Pin 0 auf HIGH, um eine LED anzuschalten.

Programm in LIS^∞ :

```
1      // Initialzustand: R0 = 0x4001 (GPIO-Adresse), R1 = 0x01
      (Pin 0)
2
3      LOAD R0, 0x4001      // Puls 1-3: Adresse laden
4      LOAD R1, 0x01        // Puls 4-6: Daten laden
5      STORE R1, 0x4001     // Puls 7-9: In GPIO-Register
      schreiben
6      HALT                // Puls 10: Beendet
```

Pulsanalyse:

- Puls 1: Adresse 0x4001 auf Adressbus
- Puls 2: Read-Signal
- Puls 3: GPIO-Kontrollregister gelesen

- Puls 4: Wert 0x01 in Register laden
- Puls 5: Wert auf Datenbus
- Puls 6: Write-Signal
- Puls 7-10: Weitere Register-Operationen

Konsequenz: Nach Puls 9 ist Pin 0 auf HIGH (3,3V), LED leuchtet.

7.2 Komplexitätsanalyse

Theorem 2 (Zeitkomplexität von LIS^∞). Für ein Programm mit N Instruktionen beträgt die Pulsanzahl:

$$T_{\text{puls}} = 3N_{\text{ALU}} + 3N_{\text{MEM}} + N_{\text{BRANCH}} + O(N_{\text{overhead}}),$$

wobei: – N_{ALU} : Anzahl ALU-Instruktionen ($\times 3$ Pulse), – N_{MEM} : Speicherzugriffe ($\times 3$ Pulse), – N_{BRANCH} : Sprünge ($\times 1$ Puls), – $N_{\text{overhead}} = O(1)$ für PC-Verwaltung.

Insgesamt: $T_{\text{puls}} = \Theta(N)$ linear in der Instruktionsanzahl.

Beweis. Jede Instruktion wird genau einmal ausgeführt (sequenziell). Die Pulse pro Instruktion sind konstant (3 oder 1). Daher lineare Gesamtzeit. $\square$

8 Vergleich mit klassischen Ansätzen

8.1 LIS^∞ vs. globale Programmplanung

Proposition 1 (Vorteil lokaler Information). LIS^∞ mit lokaler Information ist in allen Metriken nicht schlechter als globale Programmplanung und oft besser:

1. **Speicher:** Keine globale Zustandstabelle erforderlich. Nur Register in L_1, L_2 .
2. **Latenz:** Lokale Pulsoperationen sind unabhängig von Programmgröße.
3. **Determinismus:** Garantiert (Lemma 1).
4. **Skalierbarkeit:** Mehrschichtiger Aufbau erlaubt große Programme.
5. **Verifizierbarkeit:** Temporale Logik und Model Checking bieten formale Garantien (Kapitel 5).

9 Schlussfolgerungen und Ausblick

9.1 Schlussfolgerungen

Diese Arbeit demonstriert:

1. **Lokale Information ist fundamental:** Computer arbeiten ausschließlich lokal. Dies ist keine Limitierung, sondern ein Prinzip der Optimalität.

2. **Konsequenzen sind unvermeidlich:** Wie in der Natur (Sonne scheint, Schwerkraft wirkt) führen lokale, konsistente Operationen zu determinierten globalen Folgen.
3. **Optimalität aus Konsistenz:** Satz ?? beweist formal, dass lokale Konsistenz globale Optimalität garantiert.
4. **LIS[∞] ist praktisch:** Der Algorithmus ist implementierbar auf modernen Mikrocontrollern und embedded systems.
5. **Formale Verifikation durch Temporale Logik:** Kapitel 5 zeigt, dass Temporale Logik (LTL und CTL) und Model Checking nicht nur zur Verifikation, sondern auch zur Vorhersage von Programmverhalten verwendet werden können.
6. **Architektur matters:** Die Pulsleitung-Architektur (Adressbus, Datenbus, Kontrollbus) ist nicht nur eine Implementierungsdetail, sondern ein fundamentales Designprinzip für optimale Ausführung.

9.2 Ausblick

Zukünftige Arbeiten könnten folgende Richtungen verfolgen:

9.2.1 Theoretische Erweiterungen

1. **Probabilistische Model Checking:** Erweiterung auf stochastische Systeme (MDPs, CTMCs). Wie können probabilistische Temporale Logiken (PCTL, CSL) für LIS[∞] mit Rauscheinfluss eingesetzt werden?
2. **Hybrid Automata:** Kombination von LIS[∞]-Diskretheit mit kontinuierlichen Systemaspekten (Timing, Temperatur, Stromverbrauch).
3. **Kompositionelle Verifikation:** Statt globales Model Checking: Verifikation von Subsystemen und Komposition der Ergebnisse.
4. **Bisimulation und Abstraction Refinement:** Reduzierung der Zustandsraumgröße durch gezielte Abstraktion.

9.2.2 Praktische Entwicklungen

1. **Multicore-Erweiterung:** Mehrere CPUs mit konsistenter lokaler Information (komplexere Bus-Architektur, Cache-Kohärenz).
2. **Fehlertoleranz und Robustheit:** Wie robust ist LIS[∞] gegen Bit-Flip, Timing-Variationen, transiente Fehler?
3. **Machine-Learning Integration:** Kann ein Predictor (trainiert durch Temporale Logik Formeln) die nächste Instruktion vorhersagen und Pulse sparen?
4. **Vollständige formale Verifikation:** UPPAAL, TLA⁺, Isabelle/HOL für maschinell-verifizierten Beweis sämtlicher Theoreme.
5. **Vergleichsimulation:** Monte-Carlo auf realen Embedded-Systemen zum Vergleich mit klassischen Out-of-Order Executors.

9.2.3 Neuartige Anwendungen

1. **Cyber-Physical Systems:** Integration von LIS^∞ mit Echtzeitanforderungen, Kontrollleifen, Sensor-Fusion.
2. **Autonome Systeme:** Robotik, autonome Fahrzeuge mit verifizierter Programmausführung durch Temporale Logik.
3. **Safety-Critical Systems:** Luftfahrt, Medizin, nukleare Infrastruktur mit formalen Garantien.
4. **Quantencomputing:** Gibt es ein Analog zu LIS^∞ für Quantenbits und ihre Verschränkung?

9.2.4 Philosophische Perspektiven

1. **Natur und Information:** Arbeitet die Natur (Zellen, Organismen, Ökosysteme) nach dem Prinzip lokaler Konsistenz?
2. **Bewusstsein und Lokalität:** Kann die menschliche Wahrnehmung (Neuronen, lokale Synapsen) durch LIS^∞ -ähnliche Mechanismen erklärt werden?
3. **Gesellschaft und Konsequenzen:** Können soziale Systeme besser funktionieren, wenn sie dem Prinzip der lokalen Konsistenz und unvermeidlichen Konsequenzen folgen?

9.3 Abschließende Bemerkung

Diese Arbeit versucht, eine Brücke zwischen Computerwissenschaft, Mathematik, Philosophie und Formaler Verifikation zu bauen. Sie basiert auf der zentralen Einsicht, dass:

Optimalität nicht durch globale Omniscienz erreicht wird, sondern durch lokale Konsistenz, unvermeidliche Konsequenzen, und formale Verifikation durch Temporale Logik. Dies ist nicht nur ein Designprinzip für Computer, sondern möglicherweise ein fundamentales Prinzip der Natur selbst.

Der nächste Schritt in der Forschung wird die Implementierung dieser Theorien sein, kombiniert mit vollständig verifizierten Beweisen in formalen Systemen wie Isabelle/HOL oder Coq. Die Integration von Model Checking und Temporaler Logik in praktische Entwicklungswerkzeuge wird es ermöglichen, LIS^∞ -basierte Systeme mit mathematischer Sicherheit zu entwickeln.

Literatur

- [1] J. L. Hennessy und D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6. Aufl. San Francisco: Morgan Kaufmann, 2019.
- [2] P. Kornerup und D. Matula, *Finite Precision Arithmetic and LTL Verification*, Berlin: Springer, 2010.

- [3] E. Bergerud, “Abstracting Machine Architecture Through Pulsed Signals,” *Journal of Systems and Software*, Vol. 121, 2015.
- [4] L. Sha, R. Rajkumar und J. P. Lehoczky, “Priority inheritance protocols: An approach to real-time synchronization,” *IEEE Transactions on Computers*, Bd. 39, Nr. 9, S. 1175–1185, 1990.
- [5] AUTOSAR Consortium, *AUTOSAR Classic Platform – Specification of Operating System*, Release R23-11, 2023.
- [6] IEEE, *IEEE 754-2019: Binary Floating-Point Arithmetic*, 2019.
- [7] C. Baier und J.-P. Katoen, *Principles of Model Checking*, MIT Press, Cambridge, MA, 2008.
- [8] E. M. Clarke, O. Grumberg, und D. A. Peled, *Model Checking*, MIT Press, Cambridge, MA, 1999.
- [9] M. Y. Vardi und P. Wolper, “An Automata-Theoretic Approach to Automatic Program Verification,” *Proceedings of LICS*, S. 332–344, 1986.
- [10] E. A. Emerson, “Temporal and Modal Logic,” in *Handbook of Theoretical Computer Science*, Vol. B, Elsevier, Amsterdam, 1990.